EXAM REVISION EDITION

Operating Systems Study Booklet

Cleaned, reorganized, and optimized from lecture-style notes for exam revision.

Patterns, comparisons, and exam-focused summaries

Preface

Welcome to the Operating Systems Study Booklet. This guide is a carefully reconstructed and polished version of standard lecture notes, explicitly designed to help you prepare for exams. Operating Systems often feels like an overwhelming collection of abstract definitions, hardware details, and complex algorithms. However, this subject becomes highly manageable once you step back and understand the recurring system ideasâ€”the underlying logic of resource management and protectionâ€”rather than memorizing isolated definitions.

The aim of this booklet is to provide a clean, typed, and structured approach. We have transitioned away from scattered slides into a pattern-based review guide. Each chapter highlights not just the *what*, but the *why* and the *how*, pointing out exactly when certain concepts will appear on your exams. Letâ€™s demystify the kernel and master the logic of the machine.

Table of Contents

- Chapter 1: Foundations of Operating Systems
- Chapter 2: Processes, Threads, and CPU Scheduling
- Chapter 3: Process Synchronization and Deadlocks
- Chapter 4: Memory Management and Virtual Memory
- Chapter 5: File Systems and Storage
- Chapter 6: I/O Systems, Disk Scheduling, and Protection
- Chapter 7: Distributed and Modern OS Concepts
- Final Revision Toolkit
- Practice Problems
- Model Solutions

CHAPTER 1

Foundations of Operating Systems

Overview: The Operating System (OS) is the intermediary software that manages hardware resources and provides an environment for applications to run. It acts as both a resource allocator and a control program.

When this topic appears

Expect foundational questions in multiple-choice or short-answer sections. You will often be asked to differentiate between architectural models or explain the mechanism of system calls and traps.

Core ideas and logic

- **Dual-Mode Operation:** Hardware supports at least two modes to protect the system: *User mode* (restricted) and *Kernel mode* (privileged).
- **Interrupts vs. Traps:** An interrupt is a hardware-generated signal. A trap (or exception) is a software-generated interrupt caused by an error or a specific request (system call).
- **System Calls:** The programming interface to OS services. They trigger a context switch from user to kernel mode.

OS Architecture	Characteristics	Pros / Cons
Monolithic	Entire OS runs in kernel space.	High performance / Hard to maintain, a bug can crash the whole system.
Layered	OS is divided into hierarchical layers.	Easier debugging / Lower performance due to layer-crossing overhead.
Microkernel	Moves as much as possible from kernel to user space.	Highly reliable and secure / Slower due to message-passing overhead.

EXAM LENS

Be prepared to trace the exact sequence of events when a user program invokes a system call: passing parameters (via registers, stack, or block), executing a trap instruction, switching to kernel mode, executing the routine, and returning to user mode.

KEY TAKEAWAYS

- The OS is essentially a resource allocator and control program.
- Kernel mode ensures system protection; system calls act as the secure bridge from user mode.
- Microkernels prioritize reliability, whereas monolithic kernels prioritize speed.

CHAPTER 2

Processes, Threads, and CPU Scheduling

Overview: A process is a program in execution. A thread is the smallest sequence of programmed instructions managed independently. CPU scheduling determines which process runs when multiple are ready.

When this topic appears

This is a heavy analytical topic. You will almost certainly face computational questions involving scheduling algorithms, drawing Gantt charts, and calculating turnaround or waiting times.

Core ideas and logic

- **PCB (Process Control Block):** Stores process state, program counter, CPU registers, etc.
- **Context Switch:** Saving the state of the old process and loading the state of the new one. This is pure system overhead.
- **Process vs. Thread:** Processes are isolated with their own memory; threads share memory, data, and files within the same process but have distinct registers and stacks.

Algorithm	Preemptive?	Key Trait
FCFS	No	Simple, but suffers from the Convoy Effect (short processes waiting for a long one).
SJF / SRTF	SRTF is	Provides optimal average waiting time but risks starvation and requires predicting burst times.
Round Robin	Yes	Uses a time quantum. Excellent response time, standard for time-sharing systems.

EXAM LENS

Always draw a Gantt chart first! Pay close attention to arrival times. Remember these formulas:
Turnaround Time = Completion Time - Arrival Time.
Waiting Time = Turnaround Time - Burst Time.

KEY TAKEAWAYS

- Processes own resources; threads share them.
- SJF is mathematically optimal for average wait time but practically difficult to implement perfectly.
- Round Robin balancing depends on the quantum size (too large = FCFS, too small = high context-switch overhead).

CHAPTER 3

Process Synchronization and Deadlocks

Overview: Concurrent access to shared data may result in data inconsistency (race conditions). Synchronization tools prevent this. Deadlocks occur when a set of processes are permanently blocked waiting on each other.

When this topic appears

Expect to trace pseudo-code for race conditions, compare synchronization primitives, and execute the Banker's Algorithm math for deadlock avoidance.

Core ideas and logic

- **Critical Section Requirements:** Mutual Exclusion, Progress, and Bounded Waiting.
- **Classic Problems:** Producer-Consumer, Readers-Writers, Dining Philosophers.
- **Coffman Conditions for Deadlock:** Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait. All four must hold simultaneously.

Tool	Description
Mutex Lock	Binary lock. The thread that acquires the lock must be the one to release it (ownership concept).
Semaphore	Integer variable (counting or binary) accessed via `wait()` and `signal()`. Used for signaling.
Monitor	High-level abstraction built into the programming language to simplify synchronization.

EXAM LENS

For Bankerâ€™s Algorithm problems, immediately calculate the **Need Matrix** (Max - Allocation). Then, find a safe sequence by satisfying the Need of processes using Available resources, adding their Allocation back to Available upon completion.

KEY TAKEAWAYS

- Semaphores: `wait()` decrements, `signal()` increments.

- Deadlock Prevention breaks at least one Coffman condition; Avoidance (Banker's) dynamically ensures the system never enters an unsafe state.

CHAPTER 4

Memory Management and Virtual Memory

Overview: Memory management organizes memory for multiple processes. Virtual memory gives the illusion of a massive memory space by mapping logical to physical addresses and swapping pages to disk.

When this topic appears

Expect address translation math, calculating Effective Access Time (EAT), and tracing Page Replacement algorithms to count page faults.

Core ideas and logic

- **TLB (Translation Lookaside Buffer):** A hardware cache for the page table.
- **Demand Paging:** Pages are brought into memory only when referenced. Missing pages cause a *page fault*.
- **Thrashing:** When a system spends more time swapping pages than executing code due to insufficient frames.

Concept	Paging	Segmentation
Division	Fixed-size blocks (Frames/Pages).	Variable-sized blocks based on user view.
Fragmentation	Internal Fragmentation (wasted space inside a frame).	External Fragmentation (wasted space between segments).
Hardware usage	Page table maps page number to frame number.	Segment table stores base and limit registers.

EXAM LENS

Address calculation: *Physical Address = (Frame Number Ã— Page Size) + Offset*. Remember that the offset is exactly the same in both the logical and physical address. Extract the offset bits using log2(Page Size).

KEY TAKEAWAYS

- FIFO page replacement suffers from Belady's Anomaly (more frames = more faults).

- LRU is a great approximation of Optimal but requires hardware support (counters/stack).

CHAPTER 5

File Systems and Storage

Overview: The file system maps logical file structures onto physical storage devices, managing data retrieval, free space, and metadata.

When this topic appears

Usually tested conceptually. Be ready to compare file allocation methods and explain UNIX inodes.

Core ideas and logic

- **Free Space Management:** Bit vectors or linked lists.
- **Journaling (Log-structured):** Records metadata updates to a log before applying them, ensuring fast crash recovery.

Allocation Method	Mechanism	Pros / Cons
Contiguous	Each file occupies a set of contiguous blocks.	Fast sequential access / Severe external fragmentation.
Linked	Each block contains a pointer to the next block.	No external fragmentation / Poor random access.
Indexed (inodes)	An index block brings all pointers together.	Fast direct access, supports large files / Index block overhead.

EXAM LENS

To calculate maximum file size for an inode: sum the direct blocks, plus (pointers per block Ã— block size) for single indirect, plus (pointersÂ² Ã— block size) for double indirect.

KEY TAKEAWAYS

- Contiguous is best for read-only sequential access.

- Indexed allocation (like UNIX inodes) is the modern standard balancing direct access and fragmentation.

CHAPTER 6

I/O Systems, Disk Scheduling, and Protection

Overview: Managing I/O hardware efficiently is critical. Disk scheduling minimizes head movement latency. Protection mechanisms control access to system resources.

When this topic appears

Calculating total head movement (cylinders crossed) using disk scheduling algorithms is a staple exam question.

Core ideas and logic

- **DMA (Direct Memory Access):** Bypasses the CPU for bulk data transfers, reducing interrupts to one per block rather than one per byte.
- **Spooling:** Holding output for a device that can serve only one request at a time (e.g., a printer queue).

Disk Scheduling Algorithm	Behavior
FCFS	Serves requests in arrival order. Often inefficient.
SSTF	Shortest Seek Time First. Can cause starvation of distant requests.
SCAN / C-SCAN	Elevator algorithm. Sweeps back and forth (or one-way for C-SCAN). Fairer than SSTF.
LOOK / C-LOOK	Like SCAN, but the head only goes as far as the last request in that direction before reversing.

EXAM LENS

When calculating head movement for SCAN, remember the head goes all the way to the end of the disk (0 or Max) before reversing. For LOOK, it stops at the last actual request.

KEY TAKEAWAYS

- DMA drastically reduces CPU utilization during heavy I/O operations.
- Protection is internal (access control lists), while Security is external (authentication, cryptography).

CHAPTER 7

Distributed and Modern OS Concepts

Overview: Computing extends beyond single-machine boundaries into distributed systems, multicore processing, and virtualization.

When this topic appears

Tested mostly as short definitions or comparisons (e.g., VMs vs. Containers) at the end of the exam.

Core ideas and logic

- **Distributed Systems:** Loosely coupled autonomous computers communicating over a network.
- **Virtualization:** A hypervisor (VMM) abstracts hardware. Type 1 runs on bare-metal; Type 2 runs on a host OS.
- **Real-Time OS:** Has strict timing constraints. Hard real-time guarantees deadlines; soft real-time prioritizes critical tasks.

Feature	Virtual Machines (VMs)	Containers
Virtualization Level	Hardware level (requires Guest OS).	OS level (shares Host kernel).
Isolation	Strong isolation.	Moderate isolation (namespaces).
Resource Overhead & Startup	High overhead, slow startup (minutes).	Low overhead, fast startup (seconds).

EXAM LENS

If asked to justify using containers over VMs for microservices, emphasize the low overhead, fast startup times, and efficient density of containers.

KEY TAKEAWAYS

- Virtualization runs multiple OSs on one hardware setup; Containers run multiple isolated user-spaces on one OS.

- Real-Time OSs are defined by determinism, not just raw speed.

Final Revision Toolkit

Problem Recognition Checklist

- **"Calculate average waiting time..."** â†’ Draw a Gantt chart. Check if preemptive or non-preemptive.
- **"System has X resources and Y processes..."** â†’ Set up Banker's Algorithm matrices: Allocation, Max, Need, Available.
- **"Logical address space of X pages..."** â†’ Extract bits for page number (log2 of pages) and offset (log2 of page size).
- **"Calculate total head movement..."** â†’ Draw a number line for disk cylinders, trace the path, sum the absolute differences.

Formula and Metric Snapshot

Turnaround Time	Completion Time - Arrival Time
Waiting Time	Turnaround Time - Burst Time
Effective Access Time (EAT)	(Hit Ratio Ã— TLB Access Time) + (Miss Ratio Ã— Memory Access Time)
Physical Address	(Frame Number Ã— Page Size) + Offset

Page Replacement & Deadlock Strategy Snapshot

- **FIFO:** Simplest, but Belady's Anomaly risk. **LRU:** Replaces longest unused, great performer. **Optimal:** Looks to future, theoretical baseline.
- **Deadlock Prevention:** Break a Coffman condition. **Avoidance:** Banker's algorithm (safe states). **Detection:** Let it happen, recover later.

Practice Problems

1. Three processes arrive at time 0 with burst times P1=24, P2=3, P3=3. Calculate the average waiting time using FCFS scheduling.
2. For the same processes in Q1, calculate average waiting time using Round Robin with a time quantum of 4.
3. A system has 12 magnetic tape drives and 3 processes: P0 (needs 10, holds 5), P1 (needs 4, holds 2), P2 (needs 9, holds 2). Is the system in a safe state?
4. Explain the functional difference between a binary semaphore and a mutex lock.
5. A system uses 32-bit logical addresses and a 4KB page size. How many bits are used for the page number, and how many for the page offset?
6. Calculate the physical address if the Logical Address is 5000, Page Size is 4096 bytes, and Page 1 maps to Frame 5.
7. Given the reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. How many page faults occur with FIFO replacement using 3 frames?
8. Compare internal and external fragmentation. Give an example of an allocation scheme that suffers from each.
9. A disk queue requests cylinders: 98, 183, 37, 122, 14, 124, 65, 67. The head starts at 53. Calculate total head movement using FCFS.
10. Why is DMA used, and how exactly does it save CPU cycles?

Model Solutions

1. **FCFS Wait Times:** P1=0, P2=24, P3=27. Average = (0+24+27)/3 = 17.
2. **RR (Q=4) Wait Times:** Gantt: P1(4), P2(3), P3(3), P1(20). P2 finishes at 7 (Wait=7-3=4). P3 finishes at 10 (Wait=10-3=7). P1 finishes at 30 (Wait=30-24=6). Average = (6+4+7)/3 = 5.66.
3. **Banker's Algorithm:** Total drives=12. Allocated=9. Available=3. Needs: P0=5, P1=2, P2=7. Sequence: P1 can run (Need 2 â‰¤ Avail 3). P1 finishes, returns 2 (Avail=5). P0 can run (Need 5 â‰¤ Avail 5). P0 finishes, returns 5 (Avail=10). P2 runs. Yes, safe sequence exists (P1, P0, P2).
4. **Mutex vs Semaphore:** A mutex implies ownership (the thread that locks it must unlock it). A binary semaphore does not enforce ownership and can be signaled by a different thread, making it ideal for event signaling.
5. **Paging Math:** 4KB = 4096 bytes = 2^12. Therefore, 12 bits for offset. 32 - 12 = 20 bits for the page number.
6. **Address Translation:** Logical 5000. Page = 5000 / 4096 = 1. Offset = 5000 % 4096 = 904. Frame 5 is at 5 Ã— 4096 = 20480. Physical Address = 20480 + 904 = 21384.
7. **FIFO Faults:** Trace manually: 1(f), 2(f), 3(f), 4(f-replaces 1), 1(f-replaces 2), 2(f-replaces 3), 5(f-replaces 4), 1(hit), 2(hit), 3(f-replaces 1), 4(f-replaces 2), 5(hit). Total faults = 9.
8. **Fragmentation:** Internal is wasted space inside an allocated block (occurs in Paging). External is wasted space between allocated blocks (occurs in Contiguous allocation or Segmentation).
9. **FCFS Disk:** Path: 53 â†’ 98 â†’ 183 â†’ 37 â†’ 122 â†’ 14 â†’ 124 â†’ 65 â†’ 67. Absolute differences: 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640 cylinders.
10. **DMA:** Direct Memory Access allows I/O devices to transfer large blocks of data directly to memory. The CPU only receives one interrupt per block rather than one interrupt per byte (programmed I/O), freeing the CPU for other tasks.
